ROUX Clément
Candidat N°33830

Modélisation probabiliste et interprétation des cycles du jeu Monopoly



Publicité 2018
Monopoly Tricheur

THÈME: CYCLES, BOUCLES
TIPE 2025-2026

Introduction



_ +350 000 000 exemplaires

_ Basé sur le lancer de dés

_ Semble guidé par le hasard mais des règles spécifiques bouleversent cela.

Problématique

Comment prévoir les positions préférentiels des joueurs afin d'augmenter les probabilités de victoire?

Sommaire

_ Modélisation et hypothèses

_ Etude mathématique

_ Matrice de transition

_ Distribution stationnaire

_ Evaluation des stratégies

_ Annexe

Hypothèses

_ Partie classique (2 dés)

_ Dés équilibrés

_ Aucune triche

_ Aucun usage de la carte sortir de prison

_ Ne paye pas pour sortir de prison

_ Regroupe les cartes sortie de prison et gagner ou perdre de l'argent

Implémentation

_ E l'ensemble des états possibles correspondant à la position du joueur indicée par son nombre de double (variant de 0 à 2).

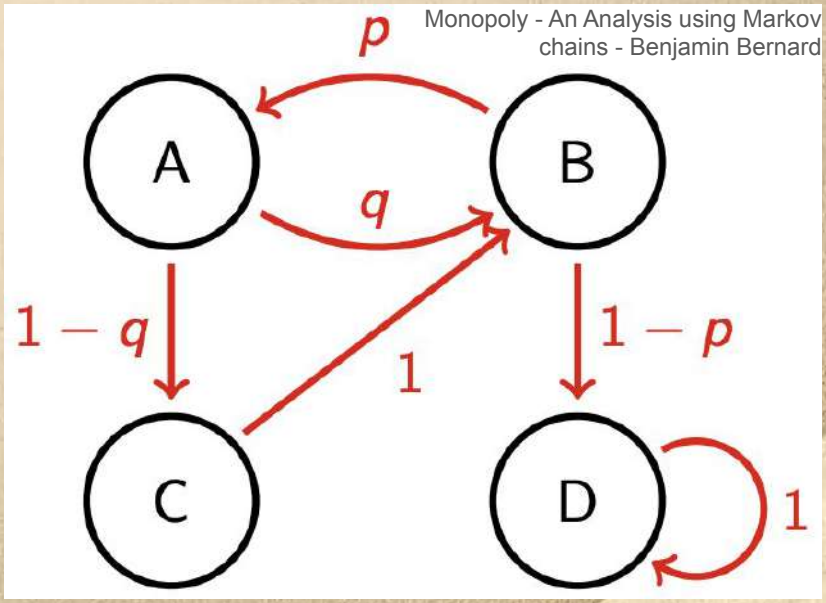
_ Card(E)=120 (119 en réalité car prison état 2 =visite simple état 0).

```
cases = [
    'Case Départ', 'Boulevard de Belleville',
    'Caisse de Communauté0', 'Rue Lecourbe', 'Impôts sur le Revenu',
    'Gare Montparnasse', 'Rue de Vaugirard', 'Chance0',
    'Rue de Courcelles', 'Avenue de la République', 'Simple Visite',
    'Boulevard de la Villette', 'Compagnie de Distribution délectricité',
    'Avenue de Neuilly', 'Rue de Paradis', 'Gare de Lyon',
    'Avenue de Mozart', 'Caisse de Communauté1',
    'Boulevard Saint-Michel', 'Place Pigalle', 'Parc Gratuit',
    'Avenue Matignon', 'Chance1', 'Boulevard Malesherbes',
    'Avenue Henri-Martin', 'Gare du Nord', 'Faubourg Saint-Honoré',
    'Place de la Bourse', 'Compagnie de Distribution des eaux',
    'Rue la Fayette', 'Prison',
    'Avenue de Breteuil', 'Avenue Foch', 'Caisse de Communauté2',
    'Boulevard des Capucines', 'Gare Saint-Lazare', 'Chance2',
    'Avenue des Champs-Elysées', 'Taxe de Luxe', 'Rue de la Paix',

    'Case Départ état 1', 'Boulevard de Belleville état 1',
    'Caisse de Communauté0 état 1', 'Rue Lecourbe état 1', 'Impôts sur le Revenu état 1',
    'Gare Montparnasse état 1', 'Rue de Vaugirard état 1', 'Chance0 état 1',
    'Rue de Courcelles état 1', 'Avenue de la République état 1', 'Simple Visite état 1',
    'Boulevard de la Villette état 1', 'Compagnie de Distribution délectricité état 1',
    'Avenue de Neuilly état 1', 'Rue de Paradis état 1', 'Gare de Lyon état 1',
    'Avenue de Mozart état 1', 'Caisse de Communauté1 état 1',
    'Boulevard Saint-Michel état 1', 'Place Pigalle état 1', 'Parc Gratuit état 1',
    'Avenue Matignon état 1', 'Chance1 état 1', 'Boulevard Malesherbes état 1',
    'Avenue Henri-Martin état 1', 'Gare du Nord état 1', 'Faubourg Saint-Honoré état 1',
    'Place de la Bourse état 1', 'Compagnie de Distribution des eaux état 1',
    'Rue la Fayette état 1', 'Prison état 1',
    'Avenue de Breteuil état 1', 'Avenue Foch état 1', 'Caisse de Communauté2 état 1',
    'Boulevard des Capucines état 1', 'Gare Saint-Lazare état 1', 'Chance2 état 1',
    'Avenue des Champs-Elysées état 1', 'Taxe de Luxe état 1', 'Rue de la Paix état 1',

    'Case Départ état 2', 'Boulevard de Belleville état 2',
    'Caisse de Communauté0 état 2', 'Rue Lecourbe état 2', 'Impôts sur le Revenu état 2',
    'Gare Montparnasse état 2', 'Rue de Vaugirard état 2', 'Chance0 état 2',
    'Rue de Courcelles état 2', 'Avenue de la République état 2', 'Simple Visite état 2',
    'Boulevard de la Villette état 2', 'Compagnie de Distribution délectricité état 2',
    'Avenue de Neuilly état 2', 'Rue de Paradis état 2', 'Gare de Lyon état 2',
    'Avenue de Mozart état 2', 'Caisse de Communauté1 état 2',
    'Boulevard Saint-Michel état 2', 'Place Pigalle état 2', 'Parc Gratuit état 2',
    'Avenue Matignon état 2', 'Chance1 état 2', 'Boulevard Malesherbes état 2',
    'Avenue Henri-Martin état 2', 'Gare du Nord état 2', 'Faubourg Saint-Honoré état 2',
    'Place de la Bourse état 2', 'Compagnie de Distribution des eaux état 2',
    'Rue la Fayette état 2', 'Prison état 2',
    'Avenue de Breteuil état 2', 'Avenue Foch état 2', 'Caisse de Communauté2 état 2',
    'Boulevard des Capucines état 2', 'Gare Saint-Lazare état 2', 'Chance2 état 2',
    'Avenue des Champs-Elysées état 2', 'Taxe de Luxe état 2', 'Rue de la Paix état 2'
]
```

Un exemple de chaîne de Markov



$$\begin{pmatrix} 0 & q & 1-q & 0 & 0 \\ p & 0 & 0 & 0 & 1-p \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

_ Une chaîne de Markov effectue des transitions aléatoires entre les états A, B, C et D tel que la somme des probabilités partant de A, B, C ou D fait 1.

_ Le Monopoly en est une généralisation composé de 120 sommets.

Définitions et propriétés

Définition 1. Soit E un ensemble fini. Une chaîne de Markov homogène sur E est une suite de variables aléatoires $(X_n)_{n\in N}$ à valeurs dans E telle que, pour tout $k \in N$ et tous $x_0, \ldots, x_{k+1} \in E$ tels que $P(X_k = x_k, \ldots, X_0 = x_0) > 0$, on a $P(X_{k+1} = x_{k+1} \mid X_k = x_k, \ldots, X_0 = x_0) = P(X_1 = x_{k+1} \mid X_0 = x_k)$.



Définition 2. On appelle *matrice de transition* la matrice $P = (p_{x,y})_{x,y\in E}$, où $\forall x, y \in E$, $p_{x,y} = P(X_{k+1} = y \mid X_k = x)$:

$$P = \begin{pmatrix} p_{x_0,x_0} & p_{x_0,x_1} & p_{x_0,x_2} & \cdots \\ p_{x_1,x_0} & p_{x_1,x_1} & p_{x_1,x_2} & \cdots \\ p_{x_2,x_0} & p_{x_2,x_1} & p_{x_2,x_2} & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}.$$

Proposition 5. *Toute matrice de transition vérifie les propriétés suivantes :*

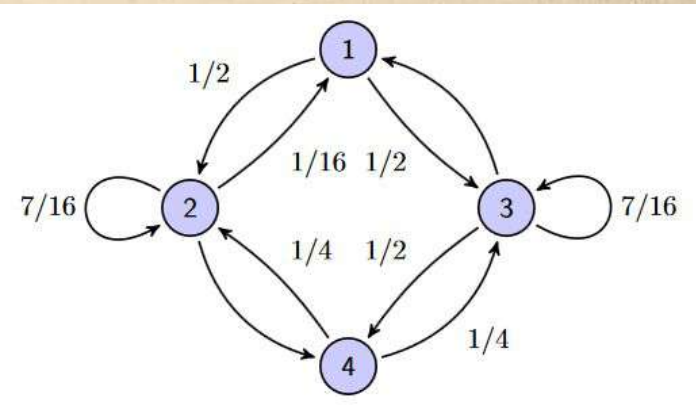
(1) *pour tout couple (x, y) de E, $0 \leq p_{x,y} \leq 1$;*

(2) *pour tout $x \in E$, on a $\sum_{y\in E} p_{x,y} = 1$.*

Une matrice qui vérifie les deux points ci-dessus est appelée *matrice stochastique.*

$\mathcal{P}$ admet 1 comme valeur propre ;

Définitions et propriétés



_ Relation d'accessibilité

_ Relation de communication est une relation d' équivalence.

Définition 19. *S'il n'y a qu'une seule classe pour la relation de communication, autrement dit, si tous les états communiquent entre eux, la chaîne est dite irréductible.*

Définitions et propriétés

Définition 20. *Soit $j \in E$. On appelle* période *de j, et on note $d(j)$, le P.G.C.D. de tous les entiers $n \geq 1$ pour lesquels $p_{j,j}^{(n)} > 0$ (par convention, $pgcd(\emptyset) = +\infty$)*

$$d(j) = pgcd(n \geq 1, p_{j,j}^{(n)} > 0)$$

Si $d(j) = d \geq 2$, on dit que j est périodique de période d. Si $d(j) = 1$, on dit que j est apériodique. *Une chaîne apériodique est une chaîne de dont tous les états sont apériodiques.*

Théorème 21.

Si i est périodique de période d finie et si $i \leftrightsquigarrow j$ $(j \neq i)$, alors j est aussi périodique de période d : la périodicité est une propriété de classe.



Définitions et propriétés

Définition 27. *Soit ν une loi de probabilité sur l'ensemble des états. Elle est dite stationnaire, ou invariante, si $\nu = \nu\mathcal{P}$ ou, de façon équivalente, si pour tout $j \in E$ on a*

$$\nu_j = \sum_{i \in E} \nu_i p_{i,j}$$

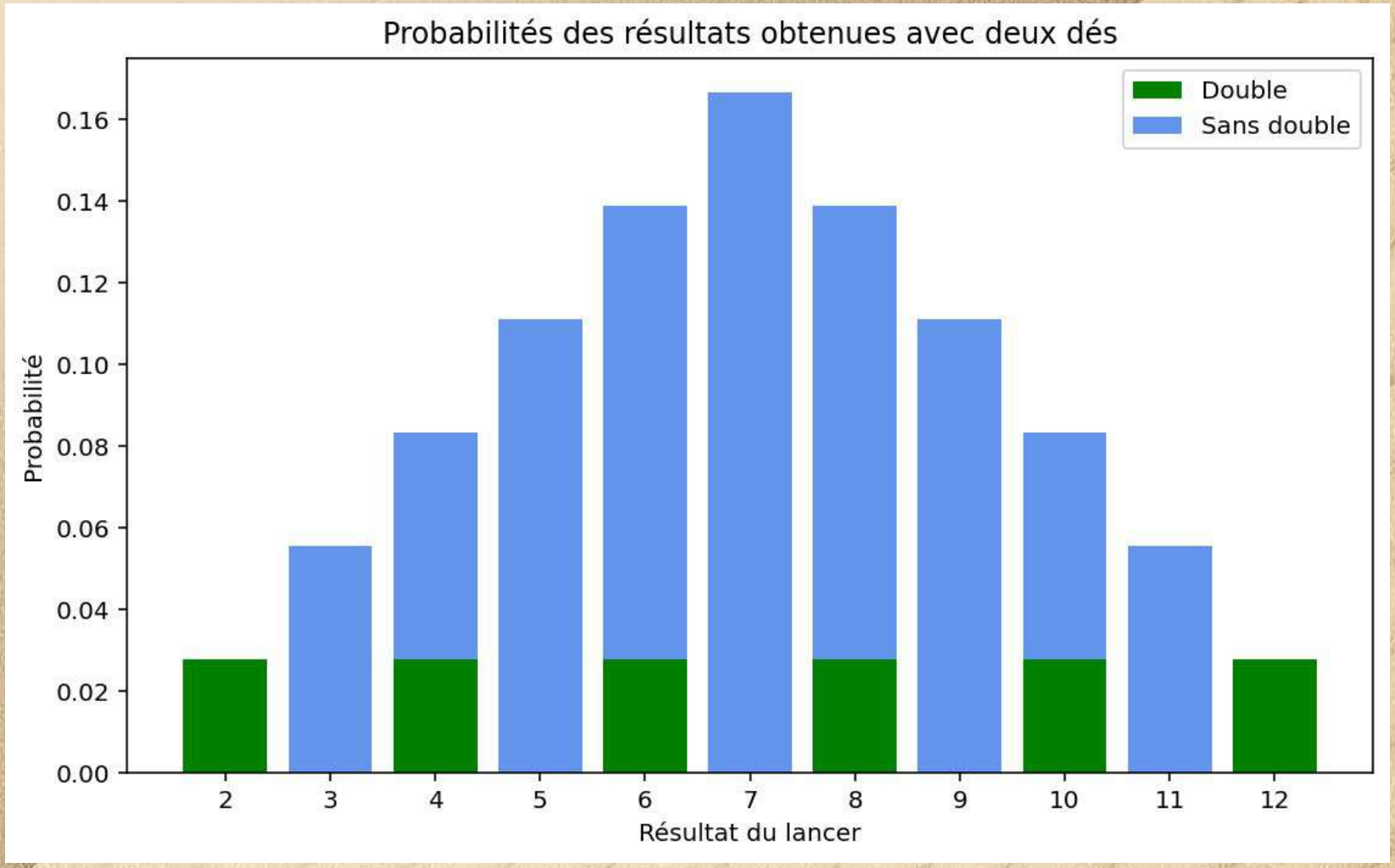
Théorème 29. Soit (X_n) une chaîne de Markov à ensemble d'états **fini**. *Alors elle possède* **au moins une loi de probabilité stationnaire**.

De plus si (X_n) est **irréductible**, *alors la loi de probabilité stationnaire est* **unique**. *On la notera ν .*

Si de plus (X_n) est **irréductible et apériodique**, *alors $\mathcal{P}^n$ converge vers la matrice dont toutes les lignes sont constantes égales à ν.*



Probabilité des dés



Matrice de transition simplifiée

Pour cette matrice:

_ on prend en compte les doubles.

_ on ne s'occupe pas des cartes spéciales.

_ on distingue les cases prison du reste.

La matrice est creuse (beaucoup de zéros) donc on calcule le dictionnaire d'adjacence puis sa matrice associée (carrée de taille 120)

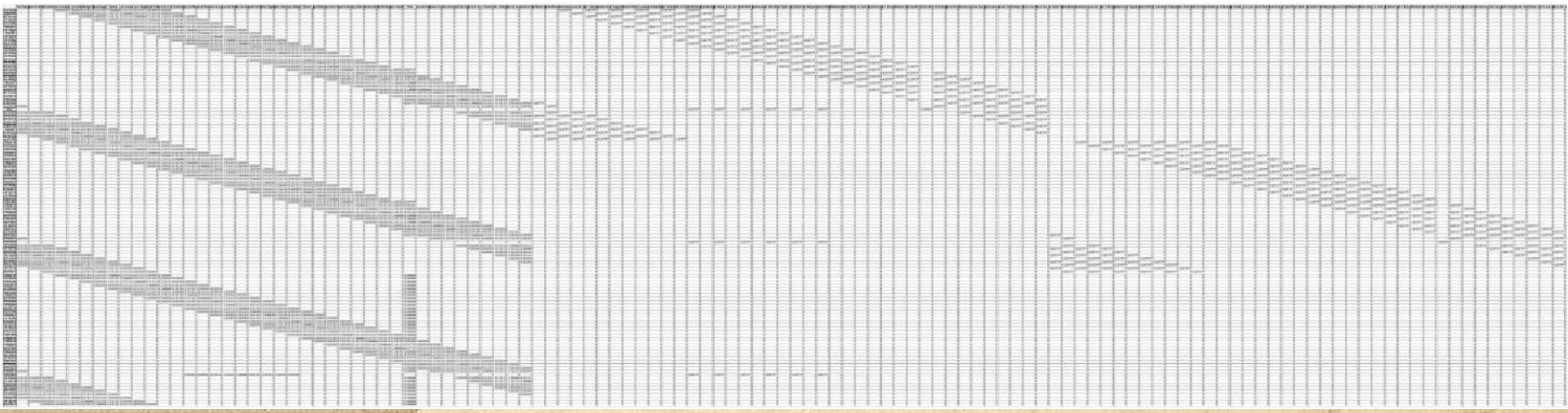
```
ajouter(dico, cle, valeur)

construire_dadj(cases, proba)

construire_matrice(cases, dadj)

verifier_stochasticite(M)
```

Matrice de transition simplifiée



Matrice de transition non simplifiée

On prend en compte les cartes spéciales.

```
ajouter(dico, cle, valeur)

gare_la_plus_proche(case_plateau)

compagnie_la_plus_proche(case_plateau)

appliquer_caisse(dico, case_plateau, p_arrivee, bloc)

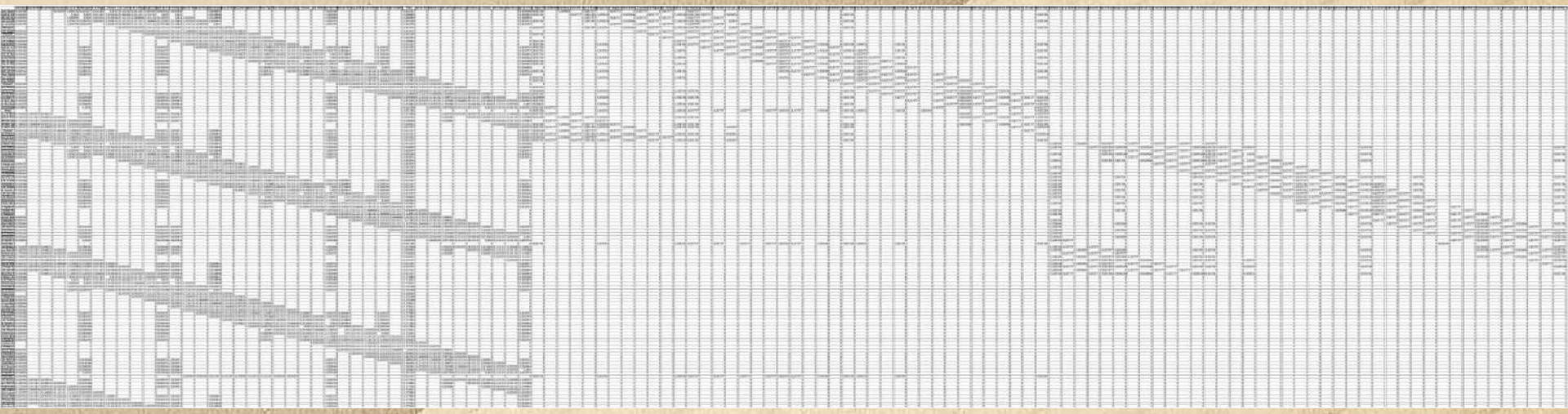
appliquer_chance(dico, case_plateau, p_arrivee, bloc)

construire_dadj(cases, proba)

construire_matrice(cases, dadj)

verifier_stochasticite(M)
```

Matrice de transition



Propriétés de la matrice de transition

_ Diagonalisable

_ 1 est valeur propre unique et 1 est le rayon spectral.
=> garantit la convergence vers la distribution stationnaire.

_ Vitesse de convergence donnée par la seconde valeur propre de module max = 0,871.
=> erreur décroît de 0,871^n par tour de jeu

Précision souhaitée	Tours de jeu	Tours de plateau
10%	17	3 tours
1%	34	6 tours
0.1%	51	9 tours
0.01%	68	12 tours

Calcul de la distribution stationnaire

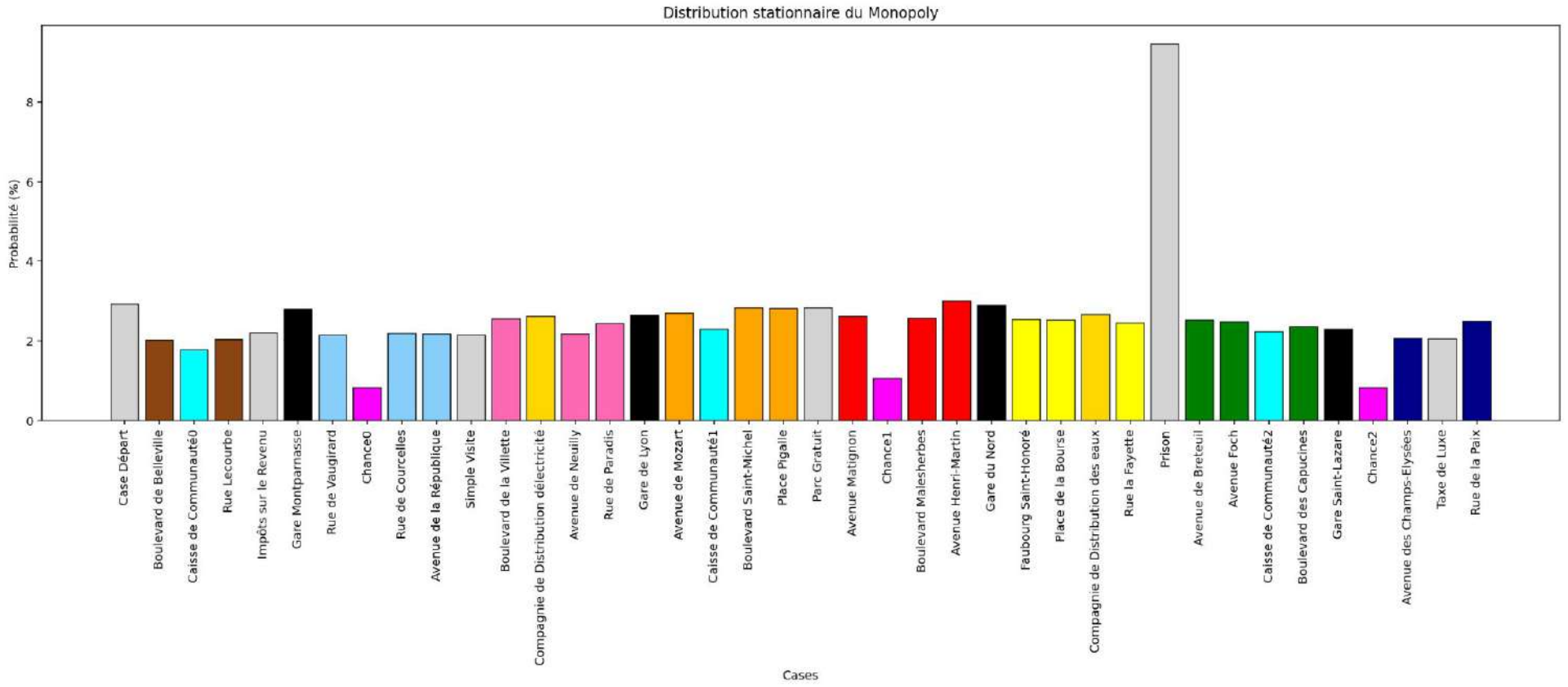
Plusieurs calculs possibles:

- $\pi^{(n+1)} = \pi^{(n)} P$ donc par récurrence $\pi = \pi_0 \lim_{n \to +\infty} P^n$ et comme P est DZ, on a $\pi = \pi_0 T \left(\lim_{n \to +\infty} D^n \right) T^{-1}$.

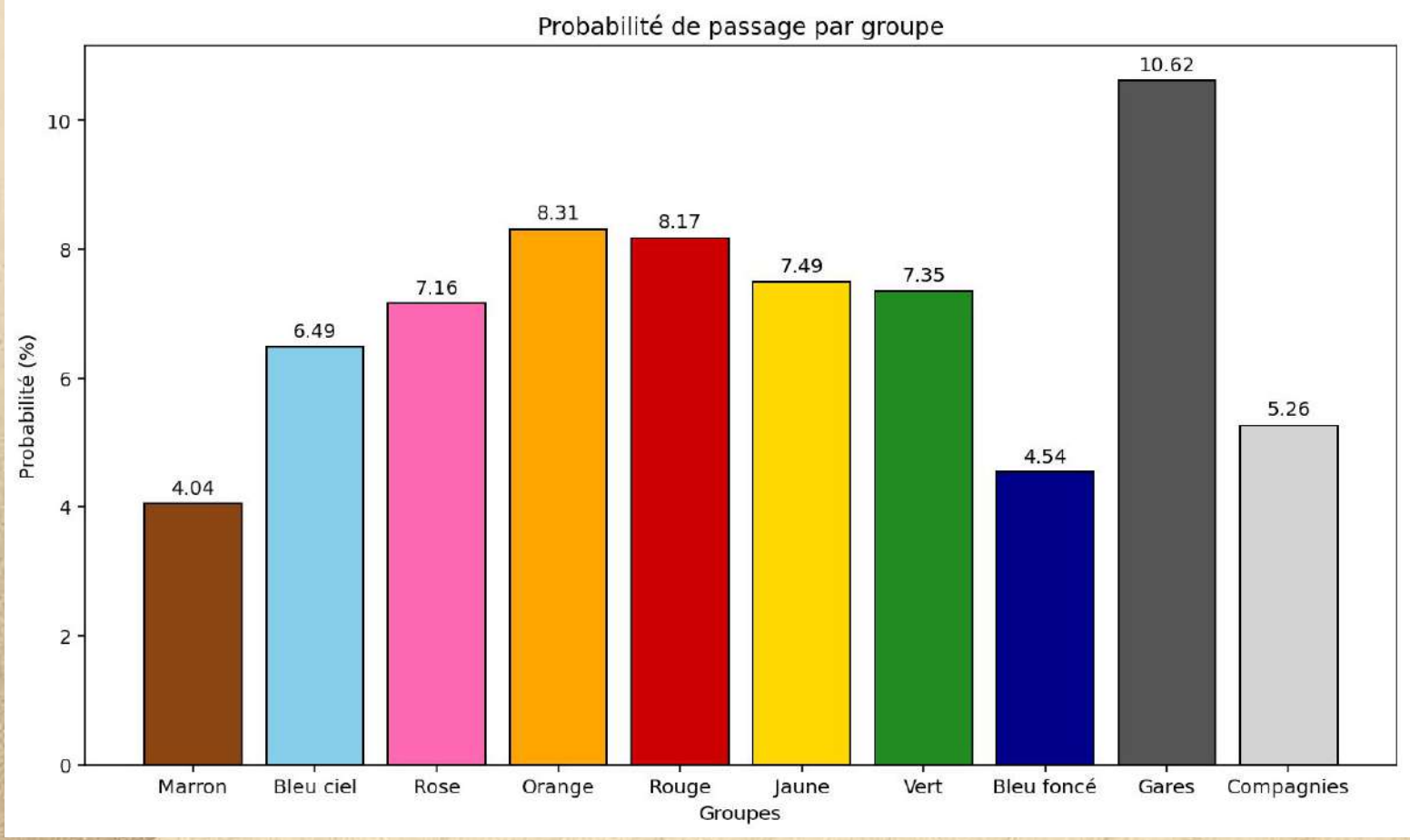
- $\pi P = \pi$ qui équivaut à $(P^\top - I)\pi^\top = 0$

=> Choix retenu: le 2 car plus précis et algorithme avec une plus petite complexité.

Distribution stationnaire

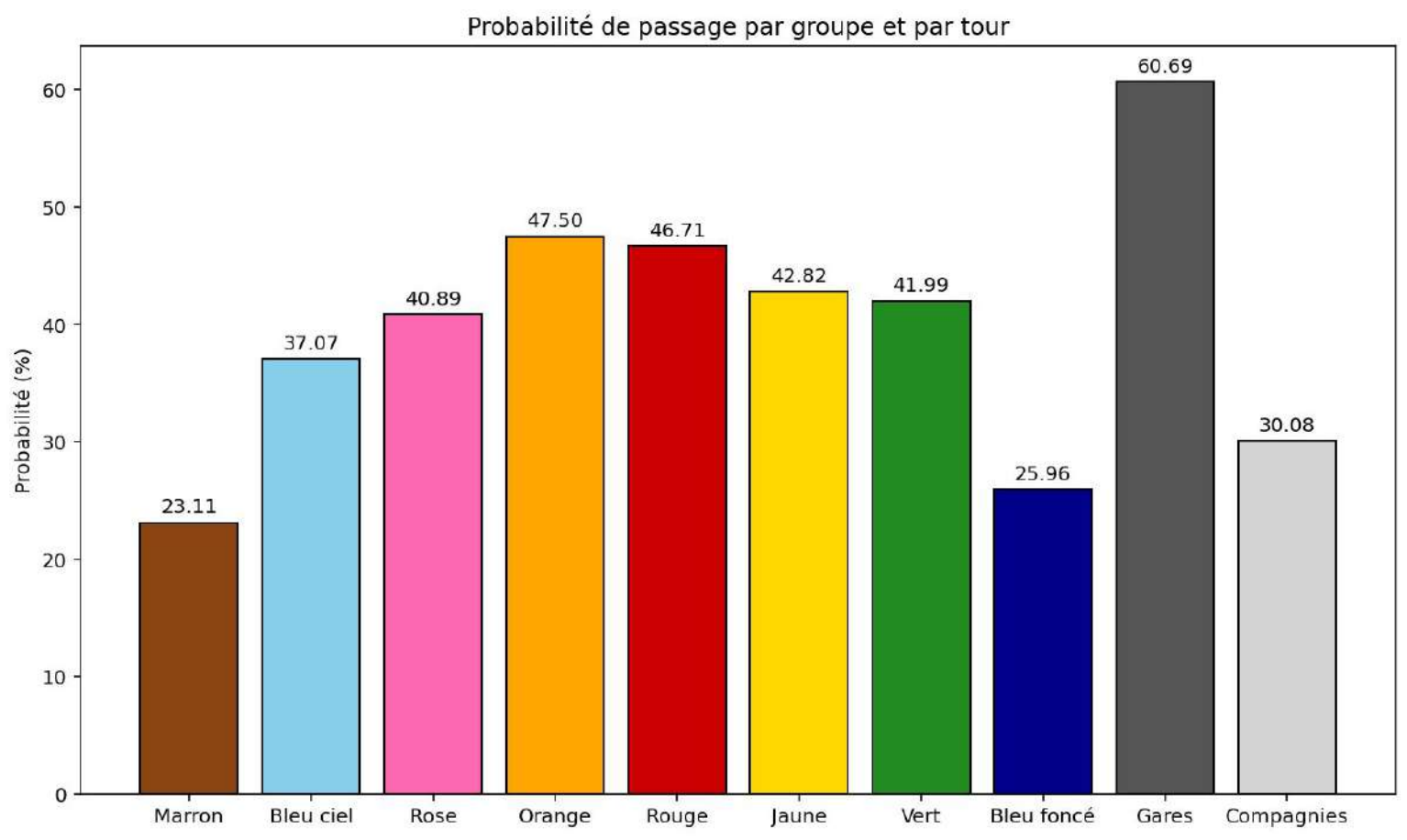


Distribution stationnaire



Cohérent avec les cycles créés par les différents renvois à la case prison.

Fréquentation des groupes par tour de plateau



On multiplie par le nombre de tour moyen: 40/7 (40 cases et le nombre moyen des dés est 7).

Retour sur Investissement

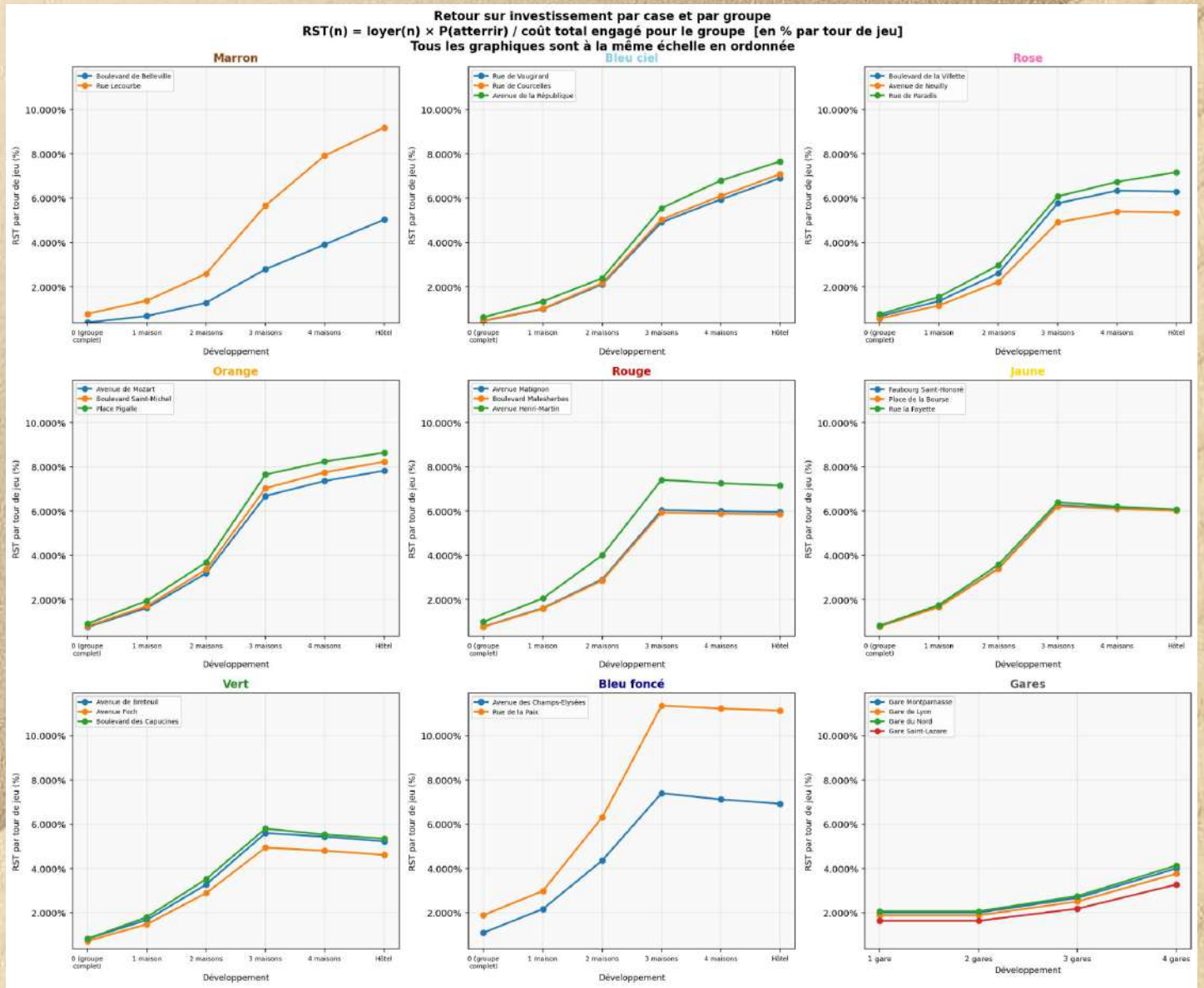
On définit le Retour Sur Investissement par tour de plateau par

$$RST = \frac{40 \times \text{proba} \times \text{loyer}}{7 \times (\text{prix d'achat et de developpement})}$$

Poser une maison, il faut que les autres cases du groupe aient le même nombre de maison ou une de plus.

```
calculer_rst(groupe_nom, cases_groupe)
```

Rentabilité des hôtels



Optimisation des stratégies

_ Acheter les couleurs les plus fréquentés(orange, rouge,jaune) qui avantageront le joueur et permettront de bons échanges.

_ Acheter les couleurs avec une meilleure rentabilité(orange, bleu foncé, bleu ciel).

_ Ne pas acheter plus de maison que le celui pour lequel le max du RST est atteint.

_ Rester le plus longtemps en prison en fin de partie et faire le contraire en début de partie.

Annexe 1.1: Démonstrations

Proposition 5. *Toute matrice de transition vérifie les propriétés suivantes :*
(1) *pour tout couple* (x, y) *de* E, $0 \leq p_{x,y} \leq 1$;
(2) *pour tout* $x \in E$, *on a* $\sum_{y \in E} p_{x,y} = 1$.

Démo: Le premier point est évident car les nombres $p_{x,y}$ sont des probabilités. Le second découle du fait qu' somme les probabilités sur toutes les valeurs possibles d'une variable aléatoire.

$\mathcal{P}$ *admet* 1 *comme valeur propre* ;

Démo: Il suffit de multiplier par le vecteur colonne qui a tous ses coefficients égaux à 1.

Théorème 21.

Si i *est périodique de période* d *finie et si* $i \leftrightsquigarrow j$ $(j \neq i)$, *alors* j *est aussi périodique de période* d : *la périodicité est une propriété de classe.*

DÉMONSTRATION. Si $i \leftrightsquigarrow j$, alors il existe deux entiers n et m tels que $p_{i,j}^{(n)} > 0$ et $p_{j,i}^{(m)} > 0$. Comme i est de période $d(i) = d$, il existe un entier $s \geq 1$ (s est un multiple de $d \geq 1$) tel que $p_{i,i}^{(s)} > 0$. On a donc $p_{j,j}^{(m+s+n)} \geq p_{j,i}^{(m)} p_{i,i}^{(s)} p_{i,j}^{(n)} > 0$. Comme $p_{i,i}^{(s)} > 0$ entraîne $p_{i,i}^{(2s)} > 0$, on a aussi $p_{j,j}^{(m+2s+n)} > 0$. La période de j divise donc à la fois $m + s + n$ et $m + 2s + n$, donc aussi leur différence s. En particulier elle divise la période $d(i)$ de i. De la même façon on montre que $d(i)$ divise $d(j)$ et donc que $d(i) = d(j)$. □

Annexe 1.2: Démonstrations

Théorème 1. *Soit $(X_n)_{n\geq 0}$ une chaîne de Markov à espace d'états fini de matrice de transition P. Alors elle possède au moins une loi stationnaire. De plus, si la chaîne est irréductible, cette loi stationnaire est unique. Enfin, si la chaîne est irréductible et apériodique, alors*

$$P^n \longrightarrow \mathbf{1}\,\nu,$$

où ν désigne l'unique loi stationnaire et $\mathbf{1}\,\nu$ la matrice dont toutes les lignes sont égales à ν.

Démonstration. **Existence.** Comme P est une matrice stochastique, on a $P\mathbf{1} = \mathbf{1}$. Ainsi 1 est une valeur propre de P et donc également de P^T. Il existe alors un vecteur non nul $u = (u_1, \ldots, u_N)^T$ tel que

$$P^T u = u.$$

Posons $v_i = |u_i|$. Pour tout i,

$$\sum_{j=1}^{N} p_{ji} v_j - v_i = \sum_{j=1}^{N} p_{ji}|u_j| - |u_i| \geq \left|\sum_{j=1}^{N} p_{ji} u_j\right| - |u_i| = 0.$$

En sommant sur i, on obtient

$$\sum_{i=1}^{N} \left(\sum_{j=1}^{N} p_{ji} v_j - v_i\right) = 0.$$

Tous les termes étant positifs ou nuls et leur somme étant nulle, ils sont tous nuls. Ainsi,

$$P^T v = v.$$

En normalisant v, on définit

$$\nu_i = \frac{v_i}{\sum_{k=1}^{N} v_k},$$

et l'on obtient une loi de probabilité vérifiant

$$P^T \nu = \nu.$$

Donc ν est une loi stationnaire.

Unicité dans le cas irréductible. Soit u un vecteur propre de P^T associé à la valeur propre 1. Posons

$$M = \max_{1\leq i\leq N} u_i$$

et choisissons i_0 tel que $u_{i_0} = M$.

Supposons qu'il existe j_0 tel que $u_{j_0} < M$. Comme la chaîne est irréductible, il existe un entier $n \geq 1$ tel que

$$p_{i_0 j_0}^{(n)} > 0.$$

Comme u est aussi vecteur propre de $(P^T)^n$,

$$u_{i_0} = \sum_{j=1}^{N} p_{i_0 j}^{(n)} u_j.$$

Or $u_j \leq M$ pour tout j et l'inégalité est stricte pour $j = j_0$. On obtient alors

$$u_{i_0} < \sum_{j=1}^{N} p_{i_0 j}^{(n)} M = M \sum_{j=1}^{N} p_{i_0 j}^{(n)} = M = u_{i_0},$$

ce qui est absurde. Ainsi toutes les composantes de u sont égales. L'espace propre associé à la valeur propre 1 est donc de dimension 1, ce qui entraîne l'unicité de la loi stationnaire après normalisation.

Convergence dans le cas irréductible et apériodique. Supposons maintenant la chaîne irréductible et apériodique. D'après le lemme précédent, il existe un entier n_0 tel que pour tout $n \geq n_0$ et tous $i, j \in E$,

$$p_{ij}^{(n)} > 0.$$

Le théorème de Perron–Frobenius implique alors que la valeur propre 1 est simple et que toute autre valeur propre λ de P vérifie

$$|\lambda| < 1.$$

Notons ν l'unique loi stationnaire. Comme

$$P\mathbf{1} = \mathbf{1} \qquad \text{et} \qquad \nu P = \nu,$$

la décomposition spectrale de P permet d'écrire

$$P^n = \mathbf{1}\,\nu + R_n,$$

où $R_n \to 0$ lorsque $n \to \infty$.

On en déduit

$$\lim_{n\to\infty} P^n = \mathbf{1}\,\nu.$$

Autrement dit,

$$\lim_{n\to\infty} p_{ij}^{(n)} = \nu_j, \qquad \forall i, j \in E.$$

La matrice limite possède donc toutes ses lignes égales à ν. En particulier, pour toute loi initiale μ,

$$\mu P^n \longrightarrow \nu.$$

Ainsi la loi de X_n converge vers l'unique loi stationnaire ν.

□

Annexe 1.2: Démonstration de la vitesse de convergence

Démonstration. Soit M une matrice stochastique irréductible, apériodique et diagonalisable. On note $\lambda_1, \dots, \lambda_{120}$ ses valeurs propres ordonnées de sorte que

$$\lambda_1 = 1 > |\lambda_2| \geq \cdots \geq |\lambda_{120}|,$$

et soit $P = (p_1, \dots, p_{120})$ une matrice de vecteurs propres à droite telle que

$$M p_i = \lambda_i p_i.$$

On note également $q_1, \dots, q_{120}$ les lignes de P^{-1}, de sorte que

$$q_i M = \lambda_i q_i, \qquad q_i p_j = \delta_{ij}.$$

Comme les vecteurs $(q_i)_{1 \leq i \leq 120}$ forment une base de $(\mathbb{R}^{120})^*$, toute distribution initiale $\pi^{(0)}$ s'écrit de manière unique

$$\pi^{(0)} = \sum_{i=1}^{120} \alpha_i q_i,$$

où les coefficients sont donnés par

$$\alpha_i = \pi^{(0)} p_i.$$

En multipliant cette décomposition par M^n, on obtient

$$\pi^{(0)} M^n = \sum_{i=1}^{120} \alpha_i q_i M^n = \sum_{i=1}^{120} \alpha_i \lambda_i^n q_i.$$

Comme $\lambda_1 = 1$, il vient

$$\pi^{(0)} M^n = \alpha_1 q_1 + \sum_{i=2}^{120} \alpha_i \lambda_i^n q_i.$$

Par ailleurs, $p_1 = \mathbf{1}$ puisque $M\mathbf{1} = \mathbf{1}$, d'où

$$\alpha_1 = \pi^{(0)} \mathbf{1} = 1.$$

La chaîne étant irréductible et apériodique, le théorème ergodique assure que

$$\pi^{(0)} M^n \longrightarrow \pi,$$

où π désigne l'unique distribution stationnaire. Comme $|\lambda_i| < 1$ pour tout $i \geq 2$, on a également

$$\pi^{(0)} M^n \longrightarrow q_1.$$

Par unicité de la limite,

$$q_1 = \pi.$$

On en déduit

$$\pi^{(0)} M^n - \pi = \sum_{i=2}^{120} \alpha_i \lambda_i^n q_i.$$

En factorisant par λ_2^n, on obtient

$$\pi^{(0)} M^n - \pi = \lambda_2^n \left(\alpha_2 q_2 + \sum_{i=3}^{120} \alpha_i \left(\frac{\lambda_i}{\lambda_2} \right)^n q_i \right).$$

Or, pour tout $i \geq 3$,

$$\left| \frac{\lambda_i}{\lambda_2} \right| < 1,$$

de sorte que

$$\left(\frac{\lambda_i}{\lambda_2} \right)^n \longrightarrow 0.$$

Le terme entre parenthèses converge donc vers $\alpha_2 q_2$ et reste borné. Il existe ainsi une constante $C > 0$ telle que

$$\| \pi^{(0)} M^n - \pi \| \leq C |\lambda_2|^n.$$

Par conséquent,

$$\| \pi^{(0)} M^n - \pi \| = O(|\lambda_2|^n).$$

De plus, lorsque $\alpha_2 \neq 0$,

$$\| \pi^{(0)} M^n - \pi \| \sim |\alpha_2| \, \|q_2\| \, |\lambda_2|^n, \qquad n \to \infty.$$

La vitesse de convergence vers la distribution stationnaire est donc gouvernée par la valeur absolue de la deuxième valeur propre. □

Annexe 2.1: Code python/ Probabilité des dés

```python
import matplotlib.pyplot as plt
import numpy as np

# Sommes possibles avec deux dés
arrivee = [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

# Probabilités des doubles
proba_double = np.array([1/36, 0, 1/36, 0, 1/36, 0, 1/36, 0, 1/36, 0, 1/36])

# Probabilités totales des sommes
proba_totale = np.array([1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1]) / 36

# Probabilités des sommes obtenues sans double
proba_non_double = proba_totale - proba_double

plt.figure(figsize=(8, 5))

# Partie verte : doubles
plt.bar(
    arrivee,
    proba_double,
    color="green",
    label="Double"
)

# Partie bleue : non-doubles
plt.bar(
    arrivee,
    proba_non_double,
    bottom=proba_double,
    color="cornflowerblue",
    label="Sans double"
)

plt.title("Probabilité des résultats obtenues avec deux dés")
plt.xlabel("Résultat du lancer")
plt.ylabel("Probabilité")
plt.xticks(arrivee)
plt.legend()

plt.tight_layout()
plt.show()
```

Annexe 2.2: Code python/ Matrice simplifiée

```python
n = len(cases)

proba = {
     'Aucun double': {2:0, 3:2/36, 4:2/36, 5:4/36, 6:4/36, 7:6/36,
                      8:4/36, 9:4/36, 10:2/36, 11:2/36, 12:0},
     'Un double':    {2:1/36, 3:0, 4:1/36, 5:0, 6:1/36, 7:0,
                      8:1/36, 9:0, 10:1/36, 11:0, 12:1/36}
 }


def ajouter(d, cle, valeur):
     if cle not in d:
         d[cle] = 0
     d[cle] += valeur
```

```python
def construire_dadj(cases, proba):
     dadj = {}

     for i in range(n):
         dadj[cases[i]] = {}
         case_plateau = i % 40
         bloc         = i // 40

         # ---- Cases normales ----
         if i != 30 and i != 70 and i != 110:
             for j in range(11):
                 s              = j + 2
                 dest_plateau = (case_plateau + s) % 40
                 p_non_double = proba['Aucun double'][s]
                 p_double     = proba['Un double'][s]

                 if p_non_double > 0:
                     if dest_plateau == 30:
                         ajouter(dadj[cases[i]], cases[30], p_non_double)
                     else:
                         ajouter(dadj[cases[i]], cases[dest_plateau], p_non_double)

                 if p_double > 0:
                     if bloc == 2:
                         ajouter(dadj[cases[i]], cases[30], p_double)
                     elif dest_plateau == 30:
                         ajouter(dadj[cases[i]], cases[30], p_double)
                     else:
                         ajouter(dadj[cases[i]], cases[dest_plateau + 40*(bloc+1)], p_double)

         # ---- Prison état 0 et 1 (30 et 70) ----
         if i == 30 or i == 70:
             for j in range(11):
                 s              = j + 2
                 p_double     = proba['Un double'][s]
                 p_non_double = proba['Aucun double'][s]
                 if p_double > 0:
                     dest_plateau = (10 + s) % 40
                     ajouter(dadj[cases[i]], cases[dest_plateau+40], p_double)
                 if p_non_double > 0:
                     if i == 30:
                         ajouter(dadj[cases[i]], cases[70],  p_non_double)
                     else:
                         ajouter(dadj[cases[i]], cases[110], p_non_double)

         # ---- Prison état 2 (110) = Simple Visite état 0 (10) ----
         if i == 110:
             dadj[cases[110]] = dadj[cases[10]]

     return dadj

dadj = construire_dadj(cases, proba)
```

```python
def construire_matrice(cases, dadj):
     M = np.zeros((n, n))
     for i in range(n):
         for j in range(n):
             if cases[j] in dadj[cases[i]]:
                 M[i][j] = dadj[cases[i]][cases[j]]
     return M

M = construire_matrice(cases, dadj)

df = pd.DataFrame(M, index=cases, columns=cases)
df.to_excel("matrice_monopoly_v3.xlsx")
print("Fichier Excel créé")

def verifier_stochasticite(M):
     problemes = [(i, sum(M[i])) for i in range(len(M)) if abs(sum(M[i]) - 1.0) > 1e-9]
     if not problemes:
         print("✓ Matrice stochastique : toutes les lignes somment à 1.")
     else:
         print(f"✗ {len(problemes)} ligne(s) non stochastique(s) :")
         for idx, s in problemes:
             print(f"  Ligne {idx:3d} ({cases[idx]}) : somme = {s:.10f}")

verifier_stochasticite(M)
```

Annexe 2.3: Code python/ Vraie matrice

```python
caisse_de_communaute = {
    'Case Départ':              1/16,
    'Allez en Prison':          1/16,
    'Gagner/Perdre de largent': 14/16
}

chance = {
    'Case Départ':                                  1/16,
    'Gare Montparnasse':                            1/16,
    'Allez en Prison':                              1/16,
    'Boulevard de la Villette':                     1/16,
    'Avenue Henri-Martin':                          1/16,
    'Rue de la Paix':                               1/16,
    'Reculez de trois cases':                       1/16,
    'Avancez jusqu à la Gare la plus proche':       2/16,
    'Avancez jusqu à la Compagnie la plus proche':  1/16,
    'Gagner/Perdre de largent':                     6/16
}

gare_cases                    = [5, 15, 25, 35]
compagnie_de_distribution_cases = [12, 28]
chance_cases                  = [7, 22, 36]
caisse_de_communaute_cases    = [2, 17, 33]
```

```python
def gare_la_plus_proche(case_plateau):
    if case_plateau == chance_cases[0]: return gare_cases[1]
    elif case_plateau == chance_cases[1]: return gare_cases[2]
    else: return gare_cases[0]

def compagnie_la_plus_proche(case_plateau):
    if case_plateau == chance_cases[0]: return compagnie_de_distribution_cases[0]
    elif case_plateau == chance_cases[1]: return compagnie_de_distribution_cases[1]
    else: return compagnie_de_distribution_cases[0]

def appliquer_caisse(d, case_plateau, p_arrivee, bloc):
    ajouter(d, cases[0  + 40*bloc], p_arrivee * caisse_de_communaute['Case Départ'])
    ajouter(d, cases[30],           p_arrivee * caisse_de_communaute['Allez en Prison'])
    ajouter(d, cases[case_plateau + 40*bloc],
                                    p_arrivee * caisse_de_communaute['Gagner/Perdre de largent'])

def appliquer_chance(d, case_plateau, p_arrivee, bloc):
    ajouter(d, cases[0  + 40*bloc],  p_arrivee * chance['Case Départ'])
    ajouter(d, cases[5  + 40*bloc],  p_arrivee * chance['Gare Montparnasse'])
    ajouter(d, cases[11 + 40*bloc],  p_arrivee * chance['Boulevard de la Villette'])
    ajouter(d, cases[24 + 40*bloc],  p_arrivee * chance['Avenue Henri-Martin'])
    ajouter(d, cases[39 + 40*bloc],  p_arrivee * chance['Rue de la Paix'])
    ajouter(d, cases[30],            p_arrivee * chance['Allez en Prison'])
    ajouter(d, cases[gare_la_plus_proche(case_plateau)       + 40*bloc],
                                     p_arrivee * chance['Avancez jusqu à la Gare la plus proche'])
    ajouter(d, cases[compagnie_la_plus_proche(case_plateau)  + 40*bloc],
                                     p_arrivee * chance['Avancez jusqu à la Compagnie la plus proche'])
    ajouter(d, cases[case_plateau    + 40*bloc],
                                     p_arrivee * chance['Gagner/Perdre de largent'])
    dest_recul = (case_plateau - 3) % 40
    if dest_recul in caisse_de_communaute_cases:
        appliquer_caisse(d, dest_recul, p_arrivee * chance['Reculez de trois cases'], bloc)
    else:
        ajouter(d, cases[dest_recul + 40*bloc], p_arrivee * chance['Reculez de trois cases'])
```

```python
def construire_dadj(cases, proba):
    dadj = {}

    for i in range(n):
        dadj[cases[i]] = {}
        case_plateau = i % 40
        bloc         = i // 40

        # ---- Cases normales ----
        if i != 30 and i != 70 and i != 110:
            for j in range(11):
                s           = j + 2
                dest_plateau = (case_plateau + s) % 40
                p_non_double = proba['Aucun double'][s]
                p_double     = proba['Un double'][s]

                if p_non_double > 0:
                    if dest_plateau == 30:
                        ajouter(dadj[cases[i]], cases[30], p_non_double)
                    elif dest_plateau in chance_cases:
                        appliquer_chance(dadj[cases[i]], dest_plateau, p_non_double, 0)
                    elif dest_plateau in caisse_de_communaute_cases:
                        appliquer_caisse(dadj[cases[i]], dest_plateau, p_non_double, 0)
                    else:
                        ajouter(dadj[cases[i]], cases[dest_plateau], p_non_double)

                if p_double > 0:
                    if bloc == 2:
                        ajouter(dadj[cases[i]], cases[30], p_double)
                    elif dest_plateau == 30:
                        ajouter(dadj[cases[i]], cases[30], p_double)
                    elif dest_plateau in chance_cases:
                        appliquer_chance(dadj[cases[i]], dest_plateau, p_double, bloc+1)
                    elif dest_plateau in caisse_de_communaute_cases:
                        appliquer_caisse(dadj[cases[i]], dest_plateau, p_double, bloc+1)
                    else:
                        ajouter(dadj[cases[i]], cases[dest_plateau + 40*(bloc+1)], p_double)
```



```python
        # ---- Prison état 0 et 1 (30 et 70) ----
        if i == 30 or i == 70:
            for j in range(11):
                s           = j + 2
                p_double     = proba['Un double'][s]
                p_non_double = proba['Aucun double'][s]
                if p_double > 0:
                    dest_plateau = (10 + s) % 40
                    if dest_plateau in chance_cases:
                        appliquer_chance(dadj[cases[i]], dest_plateau, p_double, 1)
                    elif dest_plateau in caisse_de_communaute_cases:
                        appliquer_caisse(dadj[cases[i]], dest_plateau, p_double, 1)
                    else:
                        ajouter(dadj[cases[i]], cases[dest_plateau+40], p_double)
                if p_non_double > 0:
                    if i == 30:
                        ajouter(dadj[cases[i]], cases[70],  p_non_double)
                    else:
                        ajouter(dadj[cases[i]], cases[110], p_non_double)

        # ---- Prison état 2 (110) = Simple Visite état 0 (10) ----
        if i == 110:
            dadj[cases[110]] = dadj[cases[10]]

    return dadj

dadj = construire_dadj(cases, proba)

def construire_matrice(cases, dadj):
    M = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            if cases[j] in dadj[cases[i]]:
                M[i][j] = dadj[cases[i]][cases[j]]
    return M

M = construire_matrice(cases, dadj)
```

Annexe 2.4: Code python/ Distribution stationnaire

```python
A = (M.T - np.eye(n)).copy()
A[-1, :] = 1
b        = np.zeros(n)
b[-1]    = 1

pi = np.linalg.solve(A, b)

# Vérifications
print(f"Vérification πM = π  : erreur max = {np.max(np.abs(pi @ M - pi)):.2e}")
print(f"Vérification Σπᵢ = 1 : somme      = {pi.sum():.15f}")

# retour aux 40 cases du plateau

pi_plateau = np.array([pi[k] + pi[k+40] + pi[k+80] for k in range(40)])

print(f"\n{'Rang':<5} {'Case':<45} {'Probabilité':>12} {'%':>7}")
print("-" * 72)
for rang, k in enumerate(np.argsort(pi_plateau)[::-1]):
    print(f"  {rang+1:<3} {cases[k]:<45} {pi_plateau[k]:>12.8f} {pi_plateau[k]*100:>6.2f}%")

print(f"\nSomme totale : {pi_plateau.sum():.10f}")

# export

# Matrice de transition
df_M = pd.DataFrame(M, index=cases, columns=cases)
df_M.to_excel("matrice_transition.xlsx")

# Distribution stationnaire
df_pi = pd.DataFrame({
    'Case'         : cases[:40],
    'Probabilité'  : pi_plateau,
    'Pourcentage'  : pi_plateau * 100
}).sort_values('Probabilité', ascending=False).reset_index(drop=True)
df_pi.index += 1
df_pi.to_excel("distribution_stationnaire.xlsx")

print("\nFichiers exportés : matrice_transition.xlsx, distribution_stationnaire.xlsx")
```

This is an integer

Annexe 2.5: Code python/ Tracé de la distribution stationnaire pour les propriétés

```python
couleurs_cases = []

for i, nom in enumerate(cases[:40]):
    # Groupes de propriétés
    if i in [1, 3]:
        couleurs_cases.append("saddlebrown")
    elif i in [6, 8, 9]:
        couleurs_cases.append("lightskyblue")
    elif i in [11, 13, 14]:
        couleurs_cases.append("hotpink")
    elif i in [16, 18, 19]:
        couleurs_cases.append("orange")
    elif i in [21, 23, 24]:
        couleurs_cases.append("red")
    elif i in [26, 27, 29]:
        couleurs_cases.append("yellow")
    elif i in [31, 32, 34]:
        couleurs_cases.append("green")
    elif i in [37, 39]:
        couleurs_cases.append("darkblue")
    # Gares
    elif i in [5, 15, 25, 35]:
        couleurs_cases.append("black")
    # Compagnies de distribution
    elif i in [12, 28]:
        couleurs_cases.append("gold")
    # Cartes Chance
    elif i in [7, 22, 36]:
        couleurs_cases.append("magenta")
    # Caisse de communauté
    elif i in [2, 17, 33]:
        couleurs_cases.append("cyan")
    # Toutes les autres cases
    else:
        couleurs_cases.append("lightgrey")
```

```python
plt.figure(figsize=(18,8))

plt.bar(
    cases[:40],
    pi_plateau * 100,
    color=couleurs_cases,
    edgecolor="black",
    linewidth=0.8
)

plt.xticks(rotation=90)
plt.ylabel("Probabilité (%)")
plt.xlabel("Cases")
plt.title("Distribution stationnaire du Monopoly")

plt.tight_layout()
plt.show()
```

Annexe 2.6: Code python/ Tracé de la distribution stationnaire pour les groupes

```python
donnees = {
    # --- MARRON ---
    'Boulevard de Belleville': {
        'groupe': 'Marron', 'prix_achat': 60, 'prix_maison': 50,
        'loyers': [4, 10, 30, 90, 160, 250]},
    'Rue Lecourbe': {
        'groupe': 'Marron', 'prix_achat': 60, 'prix_maison': 50,
        'loyers': [8, 20, 60, 180, 320, 450]},
    # --- BLEU CIEL ---
    'Rue de Vaugirard': {
        'groupe': 'Bleu ciel', 'prix_achat': 100, 'prix_maison': 50,
        'loyers': [12, 30, 90, 270, 400, 550]},
    'Rue de Courcelles': {
        'groupe': 'Bleu ciel', 'prix_achat': 100, 'prix_maison': 50,
        'loyers': [12, 30, 90, 270, 400, 550]},
    'Avenue de la République': {
        'groupe': 'Bleu ciel', 'prix_achat': 120, 'prix_maison': 50,
        'loyers': [16, 40, 100, 300, 450, 600]},
    # --- ROSE ---
    'Boulevard de la Villette': {
        'groupe': 'Rose', 'prix_achat': 140, 'prix_maison': 100,
        'loyers': [20, 50, 150, 450, 625, 750]},
    'Avenue de Neuilly': {
        'groupe': 'Rose', 'prix_achat': 140, 'prix_maison': 100,
        'loyers': [20, 50, 150, 450, 625, 750]},
    'Rue de Paradis': {
        'groupe': 'Rose', 'prix_achat': 160, 'prix_maison': 100,
        'loyers': [24, 60, 180, 500, 700, 900]},
    # --- ORANGE ---
    'Avenue de Mozart': {
        'groupe': 'Orange', 'prix_achat': 180, 'prix_maison': 100,
        'loyers': [28, 70, 200, 550, 750, 950]},
    'Boulevard Saint-Michel': {
        'groupe': 'Orange', 'prix_achat': 180, 'prix_maison': 100,
        'loyers': [28, 70, 200, 550, 750, 950]},
    'Place Pigalle': {
        'groupe': 'Orange', 'prix_achat': 200, 'prix_maison': 100,
        'loyers': [32, 80, 220, 600, 800, 1000]},
    # --- ROUGE ---
    'Avenue Matignon': {
        'groupe': 'Rouge', 'prix_achat': 220, 'prix_maison': 150,
        'loyers': [36, 90, 250, 700, 875, 1050]},
    'Boulevard Malesherbes': {
        'groupe': 'Rouge', 'prix_achat': 220, 'prix_maison': 150,
        'loyers': [36, 90, 250, 700, 875, 1050]},
    'Avenue Henri-Martin': {
        'groupe': 'Rouge', 'prix_achat': 240, 'prix_maison': 150,
        'loyers': [40, 100, 300, 750, 925, 1100]},
    # --- JAUNE ---
    'Faubourg Saint-Honoré': {
        'groupe': 'Jaune', 'prix_achat': 260, 'prix_maison': 150,
        'loyers': [44, 110, 330, 800, 975, 1150]},
    'Place de la Bourse': {
        'groupe': 'Jaune', 'prix_achat': 260, 'prix_maison': 150,
        'loyers': [44, 110, 330, 800, 975, 1150]},
    'Rue la Fayette': {
        'groupe': 'Jaune', 'prix_achat': 280, 'prix_maison': 150,
        'loyers': [48, 120, 360, 850, 1025, 1200]},
    # --- VERT ---
    'Avenue de Breteuil': {
        'groupe': 'Vert', 'prix_achat': 300, 'prix_maison': 200,
        'loyers': [52, 130, 390, 900, 1100, 1275]},
    'Avenue Foch': {
        'groupe': 'Vert', 'prix_achat': 300, 'prix_maison': 200,
        'loyers': [52, 130, 390, 900, 1100, 1275]},
    'Boulevard des Capucines': {
        'groupe': 'Vert', 'p                    ix_maison': 200,
        'loyers': [56, 150, 450, 1000, 1200, 1400]},
    # --- BLEU FONCÉ ---
    'Avenue des Champs-Élysées': {
        'groupe': 'Bleu foncé', 'prix_achat': 350, 'prix_maison': 200,
        'loyers': [70, 175, 500, 1100, 1300, 1500]},
    'Rue de la Paix': {
        'groupe': 'Bleu foncé', 'prix_achat': 400, 'prix_maison': 200,
        'loyers': [100, 200, 600, 1400, 1700, 2000]},
    # --- GARES ---
    'Gare Montparnasse': {
        'groupe': 'Gares', 'prix_achat': 200, 'prix_maison': None,
        'loyers': [25, 50, 100, 200]},
    'Gare de Lyon': {
        'groupe': 'Gares', 'prix_achat': 200, 'prix_maison': None,
        'loyers': [25, 50, 100, 200]},
    'Gare du Nord': {
        'groupe': 'Gares', 'prix_achat': 200, 'prix_maison': None,
        'loyers': [25, 50, 100, 200]},
    'Gare Saint-Lazare': {
        'groupe': 'Gares', 'prix_achat': 200, 'prix_maison': None,
        'loyers': [25, 50, 100, 200]}
}
```

This is an integer

```python
tours_par_plateau = 40 / 7

# Probabilités d'atterrissage par case
#converties en probabilité PAR TOUR DE PLATEAU

proba_case = {
    'Boulevard de Belleville':    pi_plateau[1]  * tours_par_plateau,
    'Rue Lecourbe':               pi_plateau[3]  * tours_par_plateau,
    'Rue de Vaugirard':           pi_plateau[6]  * tours_par_plateau,
    'Rue de Courcelles':          pi_plateau[8]  * tours_par_plateau,
    'Avenue de la République':    pi_plateau[9]  * tours_par_plateau,
    'Boulevard de la Villette':   pi_plateau[11] * tours_par_plateau,
    'Avenue de Neuilly':          pi_plateau[13] * tours_par_plateau,
    'Rue de Paradis':             pi_plateau[14] * tours_par_plateau,
    'Avenue de Mozart':           pi_plateau[16] * tours_par_plateau,
    'Boulevard Saint-Michel':     pi_plateau[18] * tours_par_plateau,
    'Place Pigalle':              pi_plateau[19] * tours_par_plateau,
    'Avenue Matignon':            pi_plateau[21] * tours_par_plateau,
    'Boulevard Malesherbes':      pi_plateau[23] * tours_par_plateau,
    'Avenue Henri-Martin':        pi_plateau[24] * tours_par_plateau,
    'Faubourg Saint-Honoré':      pi_plateau[26] * tours_par_plateau,
    'Place de la Bourse':         pi_plateau[27] * tours_par_plateau,
    'Rue la Fayette':             pi_plateau[29] * tours_par_plateau,
    'Avenue de Breteuil':         pi_plateau[31] * tours_par_plateau,
    'Avenue Foch':                pi_plateau[33] * tours_par_plateau,
    'Boulevard des Capucines':    pi_plateau[34] * tours_par_plateau,
    'Avenue des Champs-Élysées':  pi_plateau[37] * tours_par_plateau,
    'Rue de la Paix':             pi_plateau[39] * tours_par_plateau,
    'Gare Montparnasse':          pi_plateau[5]  * tours_par_plateau,
    'Gare de Lyon':               pi_plateau[15] * tours_par_plateau,
    'Gare du Nord':               pi_plateau[25] * tours_par_plateau,
    'Gare Saint-Lazare':          pi_plateau[35] * tours_par_plateau,
}



# calcul RST

groupes_ordre = [
    'Marron', 'Bleu ciel', 'Rose', 'Orange',
    'Rouge', 'Jaune', 'Vert', 'Bleu foncé', 'Gares'
]
couleurs_groupe = {
    'Marron':     '#8B4513',
    'Bleu ciel':  '#87CEEB',
    'Rose':       '#FF69B4',
    'Orange':     '#FFA500',
    'Rouge':      '#CC0000',
    'Jaune':      '#FFD700',
    'Vert':       '#228B22',
    'Bleu foncé': '#00008B',
    'Gares':      '#555555',
}
```

Annexe 2.6: Code python/ Tracé de la distribution stationnaire pour les groupes

```python
groupes = {g: [] for g in groupes_ordre}
for nom, info in donnees.items():
    if nom in proba_case:
        groupes[info['groupe']].append(nom)


proba_groupe=[pi_plateau[1]+ pi_plateau[3],
              pi_plateau[6]+ pi_plateau[8]+ pi_plateau[9],
              pi_plateau[11]+ pi_plateau[13]+ pi_plateau[14],
              pi_plateau[16]+ pi_plateau[18]+ pi_plateau[19],
              pi_plateau[21]+ pi_plateau[23]+ pi_plateau[24],
              pi_plateau[26]+ pi_plateau[27]+ pi_plateau[29],
              pi_plateau[31]+ pi_plateau[32]+ pi_plateau[34],
              pi_plateau[37]+pi_plateau[39],
              pi_plateau[5]+ pi_plateau[15]+ pi_plateau[25]+ pi_plateau[35]]


couleurs_barres = [
    couleurs_groupe['Marron'],
    couleurs_groupe['Bleu ciel'],
    couleurs_groupe['Rose'],
    couleurs_groupe['Orange'],
    couleurs_groupe['Rouge'],
    couleurs_groupe['Jaune'],
    couleurs_groupe['Vert'],
    couleurs_groupe['Bleu foncé'],
    couleurs_groupe['Gares']
]

couleurs_barres.append('#D3D3D3')
groupes_ordre.append('Compagnies')
proba_groupe.append(pi_plateau[12]+pi_plateau[28])
```

```python
fig, ax = plt.subplots(figsize=(10,6))

barres = ax.bar(
    groupes_ordre,
    np.array(proba_groupe)*100,
    color=couleurs_barres,
    edgecolor="black",
    linewidth=1
)

ax.bar_label(
    barres,
    fmt="%.2f",
    padding=3
)

plt.title("Probabilité de passage par groupe")
plt.xlabel("Groupes")
plt.ylabel("Probabilité (%)")
plt.tight_layout()
plt.show()

valeurs = np.array(proba_groupe) * (40/7)

fig, ax = plt.subplots(figsize=(10,6))

barres = ax.bar(
    groupes_ordre,
    valeurs*100,
    color=couleurs_barres,
    edgecolor="black",
    linewidth=1
)

ax.bar_label(
    barres,
    fmt="%.2f",
    padding=3
)

plt.title("Probabilité de passage par groupe et par tour")

plt.ylabel("Probabilité (%)")
plt.tight_layout()
plt.show()

couleurs_barres.pop()
groupes_ordre.pop()
proba_groupe.pop()
```

Annexe 2.6: Code python/ RST

```python
    labels_maisons = ['0 (groupe\ncomplet)', '1 maison', '2 maisons',
                      '3 maisons', '4 maisons', 'Hôtel']
    labels_gares   = ['1 gare', '2 gares', '3 gares', '4 gares']
    x_maisons      = np.arange(6)
    x_gares        = np.arange(4)

    fig, axes = plt.subplots(3, 3, figsize=(18, 15))
    fig.suptitle(
        "Retour sur investissement par case et par groupe\n"
        "RST = loyer × P(atterrir par tour de plateau) / (prix achat + n × prix maison)  [en %]",
        fontsize=14, fontweight='bold'
    )

    for ax, groupe_nom in zip(axes.flatten(), groupes_ordre):
        couleur      = couleurs_groupe[groupe_nom]
        cases_groupe = groupes[groupe_nom]
        if groupe_nom == 'Gares':
            for nom in cases_groupe:
                info  = donnees[nom]
                proba = proba_case[nom]
                rst   = [
                    info['loyers'][k-1] * proba / (k * info['prix_achat']) * 100
                    for k in range(1, 5)
                ]
                ax.plot(x_gares, rst, marker='o', label=nom, linewidth=2)
            ax.set_xticks(x_gares)
            ax.set_xticklabels(labels_gares, fontsize=8)
        else:
            for nom in cases_groupe:
                info        = donnees[nom]
                proba       = proba_case[nom]
                prix_maison = info['prix_maison']
                prix_achat  = info['prix_achat']
                roi = []
                for nb in range(6):
                    cout = prix_achat + (5 if nb == 5 else nb) * prix_maison
                    roi.append(info['loyers'][nb] * proba / cout * 100)
                ax.plot(x_maisons, roi, marker='o', label=nom, linewidth=2)
            ax.set_xticks(x_maisons)
            ax.set_xticklabels(labels_maisons, fontsize=7)

        ax.set_title(groupe_nom, fontsize=12, fontweight='bold', color=couleur)
        ax.set_ylabel("ROI par tour de plateau (%)", fontsize=9)
        ax.set_xlabel("Développement", fontsize=9)
        ax.legend(fontsize=7, loc='upper left')
        ax.yaxis.set_major_formatter(mticker.FormatStrFormatter('%.2f%%'))
        ax.grid(True, alpha=0.3)
        ax.set_facecolor('#f9f9f9')

    plt.tight_layout()
    plt.savefig("roi_par_groupe_v2.png", dpi=150, bbox_inches='tight')
    plt.show()
    print("Graphique sauvegardé : roi_par_groupe_v2.png")
```